



Kohat University of Science & Technology

Institute of Computing

Artificial Neural Networks

Assignment

Neural Network with Backpropagation for Weather Forecasting

Submitted by	Muhammad Wasiq Tanveer
Registration No.	CS120232036
Program	BSCS, 7th Semester
Subject	Artificial Neural Networks
Date	18 October 2026

GitHub repository	https://github.com/wasiqtanveer/ANN-Backpropagation-Weather-Forecasting
Dataset (CSV) source	https://storage.googleapis.com/tensorflow/tf-keras-datasets/jena_climate_2009_2016.csv.zip

1. Introduction

Forecasting means predicting a future value from what we can measure now. In this assignment I built an artificial neural network (ANN) that forecasts the air temperature 6 hours ahead using four weather readings taken at the current hour. The network is trained with the backpropagation algorithm.

I wrote the network from scratch in Python with NumPy instead of using a library like Keras or PyTorch, so that the forward pass, the backward pass (backpropagation) and the weight updates are all visible in the code.

2. Dataset

I used the Jena Climate dataset, recorded by the Max Planck Institute for Biogeochemistry at a weather station in Jena, Germany from 2009 to 2016. The original file has 14 measurements recorded every 10 minutes (about 420,000 rows).

Source: https://storage.googleapis.com/tensorflow/tf-keras-datasets/jena_climate_2009_2016.csv.zip

To keep the problem simple I prepared a smaller CSV file (weather_4params.csv) by:

- keeping only one reading per hour, which gives 70,091 rows;
- keeping only four columns: pressure, temperature, humidity and wind speed.

The first five rows of the prepared file:

Date / time	Pressure (mbar)	Temp (°C)	Humidity (%)	Wind (m/s)
01.01.2009 01:00:00	996.5	-8.05	94.4	0.21
01.01.2009 02:00:00	996.62	-8.88	93.2	0.25
01.01.2009 03:00:00	996.84	-8.81	93.5	0.18
01.01.2009 04:00:00	996.99	-9.05	92.6	0.1
01.01.2009 05:00:00	997.46	-9.63	92.2	0.4

Inputs and output

	Variable	Unit
Input 1	Air pressure	mbar
Input 2	Air temperature (now)	°C
Input 3	Relative humidity	%
Input 4	Wind speed	m/s
Output	Air temperature 6 hours later	°C

3. Methodology

3.1 Data preparation

- **Creating the target:** each row's target is the temperature 6 rows (6 hours) later. This shift is what turns the data into a forecasting problem.
- **Train / validation / test split:** the data is split in time order: the first 70% for training, the next 15% for validation and the last 15% for testing. The rows are not shuffled, so the test set really is the "future" that the network never saw.
- **Scaling:** every input is standardised (subtract the mean, divide by the standard deviation) using the training data only. Pressure is around 1000 while wind speed is around 2, so without scaling the large numbers would dominate.

3.2 Network architecture

The network has 4 input neurons, two hidden layers with 16 and 8 neurons using the tanh activation, and a single output neuron with a linear activation, since the output is a real number (a temperature).

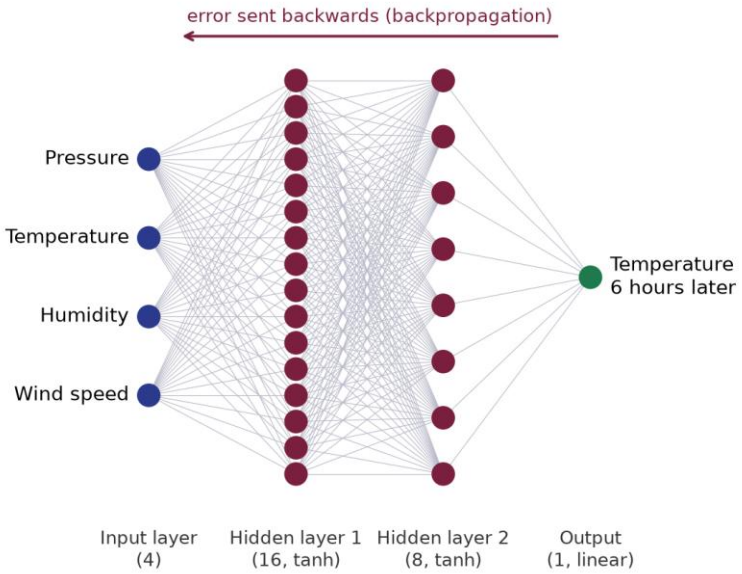


Figure 1: Structure of the network (4 → 16 → 8 → 1)

3.3 Backpropagation

Training repeats the following steps for small batches of 128 samples:

1. **Forward pass:** each layer computes $z = a \cdot W + b$ and applies the activation (tanh in the hidden layers). The last layer gives the predicted temperature.
2. **Error:** the loss is the mean squared error between the predicted and the actual temperature.
3. **Backward pass:** the error at the output is sent backwards through the network. At each layer the gradient is multiplied by the weights and by the derivative of tanh, which is $(1 - a^2)$. This gives the gradient of the loss with respect to every weight and bias.

4. **Weight update:** each weight moves a small step against its gradient. I used momentum, which keeps part of the previous step, so training is smoother and faster.

The key lines from the code that do this:

```
delta = (acts[-1] - target_s) / m                # error at the output
for i in reversed(range(len(W))):                # go backwards layer by layer
    gW[i] = acts[i].T @ delta                    # gradient for weights
    gb[i] = delta.sum(0, keepdims=True)          # gradient for biases
    if i > 0:
        delta = (delta @ W[i].T) * (1 - acts[i] ** 2)   # pass error back through tanh

vW[i] = MOM * vW[i] - LR * gW[i]                 # momentum update
W[i] += vW[i]
```

3.4 Training settings

Setting	Value
Learning rate	0.01
Momentum	0.9
Batch size	128
Epochs (iterations over the training set)	60
Loss function	Mean squared error
Weight initialisation	Xavier (random, scaled by layer size)

4. Results

4.1 Error vs iteration

The plot below shows the training and validation error before training (epoch 0) and after every epoch. The error falls sharply during the first epoch, from about 1.04 to 0.20, and after that improves only slightly. The validation curve stays close to the training curve, which means the network is not overfitting.

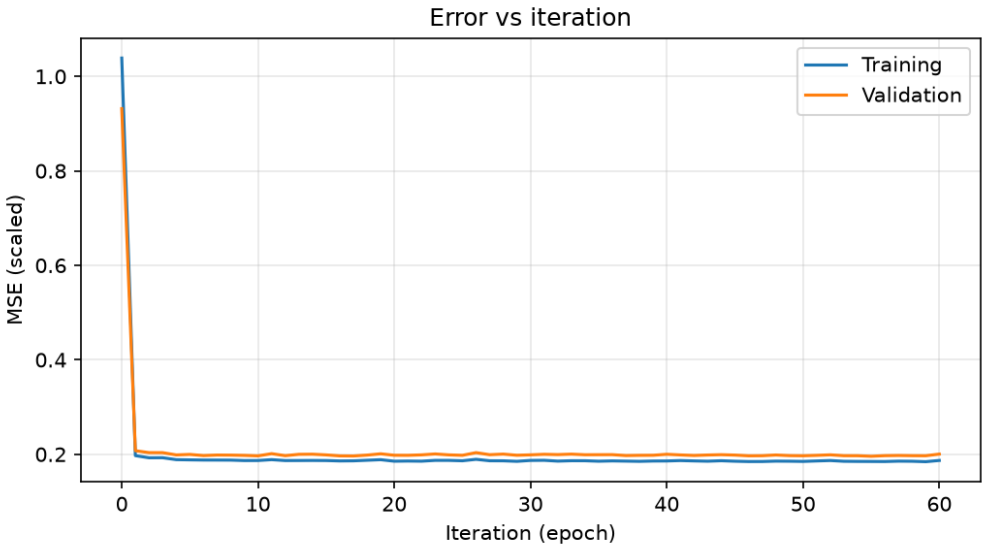


Figure 2: Error (MSE on scaled values) vs iteration

4.2 Accuracy on the test set

To judge whether the network is actually useful, I compared it with a simple baseline that assumes the temperature in 6 hours will be the same as it is now.

Model	MSE	RMSE (°C)	MAE (°C)	R^2
ANN (backprop)	12.297	3.507	2.692	0.797
Baseline (same as now)	16.117	4.015	3.013	0.734

What these measures mean:

- **MSE (Mean Squared Error):** the average of the squared errors. This is the same quantity the network minimises during training. Lower is better.
- **RMSE (Root Mean Squared Error):** the square root of MSE, so it is back in °C. It is roughly the size of a typical error.
- **MAE (Mean Absolute Error):** the average size of the error in °C, whether the forecast was too high or too low.
- **R^2 (coefficient of determination):** how much of the real temperature's variation the model explains. 1 is perfect, 0 is no better than always guessing the average.

The ANN beats the baseline on every measure. Its typical error is about 2.7 °C against 3.0 °C for the baseline, and its R^2 is 0.80 against 0.73. So the network has learned how pressure, humidity and wind affect where the temperature is heading.

4.3 Forecast vs actual temperature

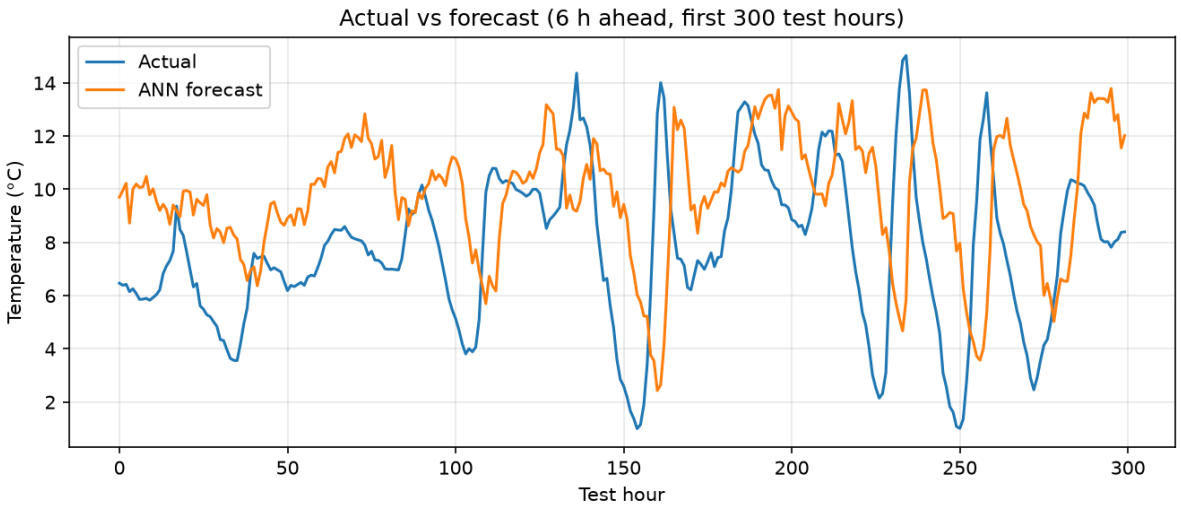


Figure 3: Actual temperature and the ANN's 6-hour forecast (first 300 test hours)

The forecast follows the daily rise and fall of the temperature. It reacts a little late to sudden drops and peaks, which is expected because the network only sees the current hour and is predicting 6 hours ahead.

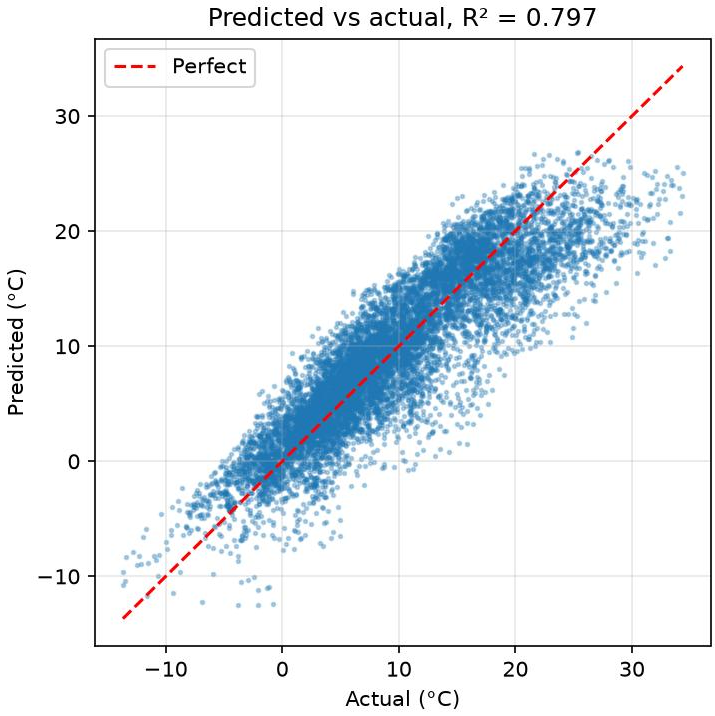


Figure 4: Predicted vs actual temperature for the whole test set

Most points lie close to the red line where prediction = actual, so the forecasts are good across the full range of test temperatures, from about −14 °C to 34 °C.

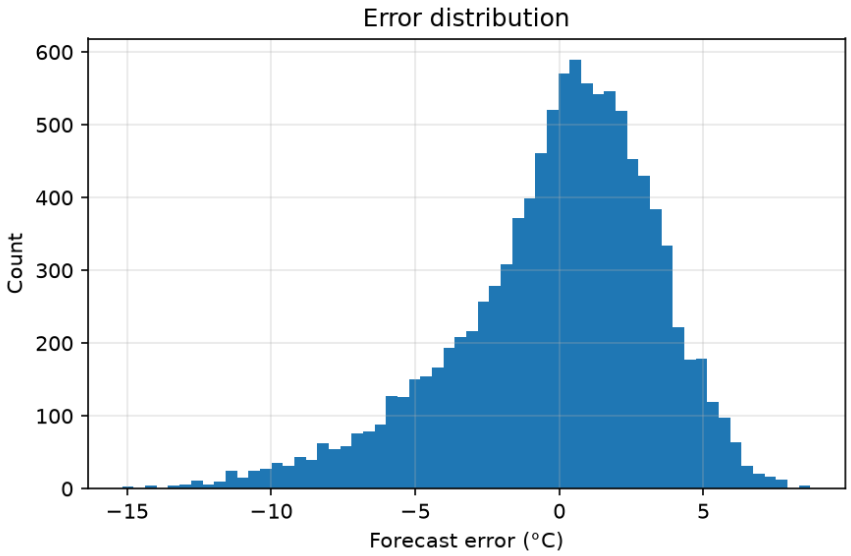


Figure 5: Distribution of forecast errors

The errors are centred near zero and are roughly bell-shaped, so the network does not consistently forecast too high or too low.

5. Conclusion

A small feed-forward neural network trained with backpropagation was able to forecast the temperature 6 hours ahead from only four current weather readings, with a typical error of about 2.7 °C. It performed better than the simple "no change" baseline. The error vs iteration curve shows that backpropagation reduced the error steadily without overfitting.

Possible improvements:

- give the network readings from the past few hours as well as the current one, so it can see trends;
- add the time of day and the time of year as inputs, since temperature follows daily and seasonal cycles;
- try a recurrent network such as an LSTM, which is designed for sequence data.

Appendix A: Program output

```
epoch   0  train MSE 1.0389  val MSE 0.9318
epoch   1  train MSE 0.1971  val MSE 0.2076
epoch  11  train MSE 0.1886  val MSE 0.2013
epoch  21  train MSE 0.1857  val MSE 0.1978
epoch  31  train MSE 0.1872  val MSE 0.2000
epoch  41  train MSE 0.1867  val MSE 0.1986
epoch  51  train MSE 0.1859  val MSE 0.1977
epoch  60  train MSE 0.1870  val MSE 0.2004

Samples: train 49059, val 10513, test 10513
Inputs (4): pressure_mbar, temp_c, humidity_pct, wind_speed_ms
Architecture: [4, 16, 8, 1], tanh hidden, linear output, lr=0.01, momentum=0.9, batch=128, epochs=60
Forecast horizon: 6 hours

Model                           MSE    RMSE     MAE      R2
ANN (backprop)               12.297   3.507   2.692   0.797
Baseline (same as now)       16.117   4.015   3.013   0.734

Saved plots and results.txt in the outputs folder
```

Appendix B: Complete source code (forecast_bp.py)

```
"""ANN with backpropagation for weather forecasting (Jena climate dataset).

Task: predict the air temperature 6 hours ahead from 4 current measurements:
pressure, temperature, humidity and wind speed.
The network is written from scratch in NumPy (forward pass, backprop, momentum SGD).
"""
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

np.random.seed(42)
HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "outputs")
os.makedirs(OUT, exist_ok=True)

# ---------------------------------------------------------------- data
df = pd.read_csv(os.path.join(HERE, "weather_4params.csv"))   # hourly readings

HORIZON = 6                                          # hours ahead to forecast
feat = df[["pressure_mbar", "temp_c", "humidity_pct", "wind_speed_ms"]].copy()
target = df["temp_c"].shift(-HORIZON)
ok = feat.notna().all(axis=1) & target.notna()
X, y = feat[ok].values, target[ok].values.reshape(-1, 1)
names = list(feat.columns)

# chronological split: 70% train / 15% validation / 15% test (no shuffling)
n = len(X)
i1, i2 = int(0.70 * n), int(0.85 * n)
Xtr, Xva, Xte = X[:i1], X[i1:i2], X[i2:]
ytr, yva, yte = y[:i1], y[i1:i2], y[i2:]

mu, sd = Xtr.mean(0), Xtr.std(0) + 1e-8            # scale using train stats only
ymu, ysd = ytr.mean(), ytr.std()
Xtr, Xva, Xte = [(a - mu) / sd for a in (Xtr, Xva, Xte)]
```

```
ytr_s, yva_s = (ytr - ymu) / ysd, (yva - ymu) / ysd

# ------------------------------------------------------------------ network
sizes = [X.shape[1], 16, 8, 1]                    # inputs, hidden1, hidden2, output
W = [np.random.randn(a, b) * np.sqrt(2.0 / (a + b)) for a, b in zip(sizes[:-1], sizes[1:])]
b = [np.zeros((1, k)) for k in sizes[1:]]
vW = [np.zeros_like(w) for w in W]
vb = [np.zeros_like(x) for x in b]


def forward(x):
    acts = [x]
    for i in range(len(W)):
        z = acts[-1] @ W[i] + b[i]
        acts.append(np.tanh(z) if i < len(W) - 1 else z)   # linear output (regression)
    return acts


def backward(acts, target_s):
    m = target_s.shape[0]
    delta = (acts[-1] - target_s) / m               # dMSE/dz for linear output (up to factor 2)
    gW, gb = [None] * len(W), [None] * len(W)
    for i in reversed(range(len(W))):
        gW[i] = acts[i].T @ delta
        gb[i] = delta.sum(0, keepdims=True)
        if i > 0:
            delta = (delta @ W[i].T) * (1 - acts[i] ** 2)   # tanh'(z) = 1 - a^2
    return gW, gb


LR, MOM, BATCH, EPOCHS = 0.01, 0.9, 128, 60
train_loss = [np.mean((forward(Xtr)[-1] - ytr_s) ** 2)]   # error before any training
val_loss = [np.mean((forward(Xva)[-1] - yva_s) ** 2)]
print(f"epoch   0  train MSE {train_loss[-1]:.4f}  val MSE {val_loss[-1]:.4f}")
for ep in range(EPOCHS):
    idx = np.random.permutation(len(Xtr))
    for s in range(0, len(idx), BATCH):
        j = idx[s:s + BATCH]
        gW, gb = backward(forward(Xtr[j]), ytr_s[j])
        for i in range(len(W)):
            vW[i] = MOM * vW[i] - LR * gW[i]
            vb[i] = MOM * vb[i] - LR * gb[i]
            W[i] += vW[i]
            b[i] += vb[i]
    train_loss.append(np.mean((forward(Xtr)[-1] - ytr_s) ** 2))
    val_loss.append(np.mean((forward(Xva)[-1] - yva_s) ** 2))
    if ep % 10 == 0 or ep == EPOCHS - 1:
        print(f"epoch {ep + 1:3d}  train MSE {train_loss[-1]:.4f}  val MSE {val_loss[-1]:.4f}")

# ------------------------------------------------------------------ evaluation
pred = forward(Xte)[-1] * ysd + ymu
persist = Xte[:, names.index("temp_c")].reshape(-1, 1) * sd[names.index("temp_c")] + mu[names.index("temp_c")]


def metrics(true, p):
    err = p - true
    return np.mean(err ** 2), np.sqrt(np.mean(err ** 2)), np.mean(np.abs(err)), \
        1 - np.sum(err ** 2) / np.sum((true - true.mean()) ** 2)


mse, rmse, mae, r2 = metrics(yte, pred)
pmse, prmse, pmae, pr2 = metrics(yte, persist)
res = (
    f"Samples: train {len(Xtr)}, val {len(Xva)}, test {len(Xte)}\n"
    f"Inputs ({len(names)}): {', '.join(names)}\n"
    f"Architecture: {sizes}, tanh hidden, linear output, lr={LR}, momentum={MOM}, "
```

```
    f"batch={BATCH}, epochs={EPOCHS}\n"
    f"Forecast horizon: {HORIZON} hours\n\n"
    f"{'Model':<28}{'MSE':>8}{'RMSE':>8}{'MAE':>8}{'R2':>8}\n"
    f"{'ANN (backprop)':<28}{mse:8.3f}{rmse:8.3f}{mae:8.3f}{r2:8.3f}\n"
    f"{'Baseline (same as now)':<28}{pmse:8.3f}{prmse:8.3f}{pmae:8.3f}{pr2:8.3f}\n"
)
print("\n" + res)
open(os.path.join(OUT, "results.txt"), "w").write(res)

# ---------------------------------------------------------------- plots
plt.figure(figsize=(7, 4))
plt.plot(range(EPOCHS + 1), train_loss, label="Training")
plt.plot(range(EPOCHS + 1), val_loss, label="Validation")
plt.xlabel("Iteration (epoch)"); plt.ylabel("MSE (scaled)")
plt.title("Error vs iteration"); plt.legend(); plt.grid(alpha=.3); plt.tight_layout()
plt.savefig(os.path.join(OUT, "1_error_vs_iteration.png"), dpi=150); plt.close()

k = 300
plt.figure(figsize=(9, 4))
plt.plot(yte[:k], label="Actual", lw=1.5)
plt.plot(pred[:k], label="ANN forecast", lw=1.5)
plt.xlabel("Test hour"); plt.ylabel("Temperature (°C)")
plt.title(f"Actual vs forecast ({HORIZON} h ahead, first {k} test hours)")
plt.legend(); plt.grid(alpha=.3); plt.tight_layout()
plt.savefig(os.path.join(OUT, "2_actual_vs_forecast.png"), dpi=150); plt.close()

plt.figure(figsize=(5, 5))
plt.scatter(yte, pred, s=3, alpha=.3)
lo, hi = yte.min(), yte.max()
plt.plot([lo, hi], [lo, hi], "r--", label="Perfect")
plt.xlabel("Actual (°C)"); plt.ylabel("Predicted (°C)")
plt.title(f"Predicted vs actual, R² = {r2:.3f}"); plt.legend(); plt.grid(alpha=.3); plt.tight_layout()
plt.savefig(os.path.join(OUT, "3_scatter.png"), dpi=150); plt.close()

plt.figure(figsize=(6, 4))
plt.hist((pred - yte).ravel(), bins=60)
plt.xlabel("Forecast error (°C)"); plt.ylabel("Count")
plt.title("Error distribution"); plt.grid(alpha=.3); plt.tight_layout()
plt.savefig(os.path.join(OUT, "4_error_hist.png"), dpi=150); plt.close()
print("Saved plots and results.txt in the outputs folder")
```